You can check whether NumPy is installed in several ways.

Method 1: From Python

Open Python and run:

```
import numpy as np

print(np.__version__)
```

If NumPy is installed, you'll see something like:

```
text
2.3.1
```

If not installed, you'll get:

```
text
ModuleNotFoundError: No module named 'numpy'
```

Method 2: From Terminal / Command Prompt

bash
```
pip show numpy
```

If installed, it will display details such as:

text
```
Name: numpy
Version: 2.x.x
Location: ...
```

If not installed:

text
```
WARNING: Package(s) not found: numpy
```

Method 3: List Installed Packages

```
bash
pip list
```

Look for `numpy` in the list.

 Install if Missing

```
bash
pip install numpy
```

Similarly, for Pandas:

```
bash
pip show pandas
```

or

```
import pandas as pd
print(pd.__version__)
```

1. NumPy

NumPy (Numerical Python) is a library used for fast numerical computations and working with arrays.

Key Concepts

* Arrays: Efficient storage of numerical data.
* Indexing & Slicing: Access specific elements or portions of an array.
* Vectorization: Perform operations on entire arrays without loops.

Example

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[2])      # Indexing
print(arr[1:4])    # Slicing
print(arr * 2)     # Vectorized operation
```

Output

```
30
[20 30 40]
[20 40 60 80 100]

# Common NumPy Methods and Functions

NumPy provides many functions for creating, manipulating, and analyzing
arrays.

 1. Creating Arrays

import numpy as np

a = np.array([1, 2, 3, 4])
b = np.zeros((2, 3))
c = np.ones((2, 2))
d = np.arange(1, 10, 2)
e = np.linspace(0, 1, 5)


| Function         | Purpose                 |
| ---------------- | ----------------------- |
```

np.array()	Create array
np.zeros()	Array of zeros
np.ones()	Array of ones
np.arange()	Range of values
np.linspace()	Evenly spaced values

---

 2. Array Information

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape)
print(arr.ndim)
print(arr.size)
print(arr.dtype)
```

Method	Description
shape	Dimensions of array
ndim	Number of dimensions
size	Total elements
dtype	Data type

---

3. Reshaping Arrays

```
arr = np.array([1, 2, 3, 4, 5, 6])

print(arr.reshape(2, 3))
print(arr.flatten())
```

Method	Description
reshape()	Change dimensions
flatten()	Convert to 1D array

---

4. Mathematical Operations

```
arr = np.array([1, 2, 3, 4])

print(np.sum(arr))
```

```
print(np.mean(arr))
print(np.max(arr))
print(np.min(arr))
print(np.std(arr))
```

Function	Description
np.sum()	Sum of elements
np.mean()	Average
np.max()	Maximum value
np.min()	Minimum value
np.std()	Standard deviation

---

5. Indexing and Slicing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])
print(arr[-1])
print(arr[1:4])
```

Output:

```
10
50
[20 30 40]
```

---

6. Vectorized Operations

```
arr = np.array([1, 2, 3, 4])

print(arr + 5)
print(arr * 2)
print(arr  2)
```

Output:

```
[6 7 8 9]
```

```
[2 4 6 8]
[ 1  4  9 16]
```

---

7. Aggregation Functions

```
arr = np.array([10, 20, 30, 40])

print(arr.sum())
print(arr.mean())
print(arr.max())
print(arr.min())
```

These methods can be called directly on arrays.

---

8. Searching and Sorting

```
arr = np.array([40, 10, 30, 20])
```

```
print(np.sort(arr))
print(np.where(arr > 20))
```

Function	Description
np.sort()	Sort array
np.where()	Find elements matching condition

---

9. Useful Statistical Functions

```
arr = np.array([10, 20, 30, 40])

print(np.median(arr))
print(np.var(arr))
print(np.std(arr))
```

Function	Description
np.median()	Middle value

| np.var() | Variance |
| np.std() | Standard deviation |

 Top 15 NumPy Functions for Interviews & Exams

1. np.array()
2. np.arange()
3. np.linspace()
4. np.zeros()
5. np.ones()
6. reshape()
7. flatten()
8. shape
9. ndim
10. sum()
11. mean()
12. max()
13. min()
14. np.sort()
15. np.where()

---

2. Pandas Basics

Pandas is a library used for data analysis and manipulation.

Main Data Structures

* Series: One-dimensional labeled array.
* DataFrame: Two-dimensional table (rows and columns).

Example: Series

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)


# Example: DataFrame


data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [23, 25, 22]
}

df = pd.DataFrame(data)
print(df)
```

Output

```
text
      Name  Age
0    Alice   23
1      Bob   25
2  Charlie   22
```

```
# Filtering Data


print(df[df['Age'] > 22])
```

Output

```
text
    Name  Age
0  Alice   23
1    Bob   25


---

 3. Data Manipulation

Data manipulation involves modifying, filtering, and transforming data
for analysis.
```

```
# Filtering


df[df['Age'] >= 23]


# Adding a New Column


df['Age_after_5_years'] = df['Age'] + 5
print(df)
```

```
# Sorting


df.sort_values('Age')


# Grouping


data = {
    'Department': ['IT', 'HR', 'IT', 'HR'],
    'Salary': [50000, 40000, 60000, 45000]
}

df = pd.DataFrame(data)

print(df.groupby('Department')['Salary'].mean())


Output

text
Department
HR    42500
```

```
IT    55000
Name: Salary, dtype: float64

#Important Pandas Methods

Pandas mainly works with Series and DataFrames.


import pandas as pd

df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [23, 25, 22],
    "Salary": [50000, 60000, 55000]
})


---

1. View Data

#head()

Shows first 5 rows by default.
```

```
print(df.head())

#tail()
Shows last 5 rows.

print(df.tail())

---
2. Information About Data
#info()

df.info()

Shows:
* Number of rows
```

* Number of columns
* Data types
* Missing values

#describe()

df.describe()

Provides statistics:

* Count
* Mean
* Standard deviation
* Min/Max
* Quartiles

---

3. Selecting Columns

print(df["Name"])

Multiple columns:

```
print(df[["Name", "Age"]])
```

---

4. Filtering Rows

```
print(df[df["Age"] > 22])
```

Output:

```
    Name  Age  Salary
0  Alice   23   50000
1    Bob   25   60000
```

---

5. Adding a Column

```
df["Bonus"] = df["Salary"] * 0.1
print(df)
```

---

6. Sorting Data

```
#Ascending

df.sort_values("Age")

#Descending

df.sort_values("Age", ascending=False)
```

---

7. Grouping Data

```
df.groupby("Age")["Salary"].mean()
```

Common aggregation functions:

```
sum()
mean()
max()
min()
count()
```

---

8. Handling Missing Values

#Check Missing Values

```
df.isnull()
```

```
#Count Missing Values

df.isnull().sum()

#Fill Missing Values

df.fillna(0)

#Remove Missing Values

df.dropna()
```

---

9. Rename Columns

```
df.rename(columns={"Salary": "Income"})
```

---

10. Delete Column

```
df.drop("Salary", axis=1)
```

axis=1 → column

axis=0 → row

---

11. Unique Values

```
df["Age"].unique()
```

Count unique values:

```
df["Age"].nunique()
```

---

12. Value Counts

```
df["Age"].value_counts()
```

Counts frequency of each value.

---

13. Data Types

```
df.dtypes
```

---

14. Read and Save Files

```
#Read CSV

df = pd.read_csv("data.csv")

#Save CSV

df.to_csv("output.csv", index=False)
```

Top Pandas Methods for Exams & Interviews

Method	Purpose
head()	First rows
tail()	Last rows
info()	Dataset information
describe()	Statistics summary
columns	Column names
shape	Rows and columns
sort_values()	Sort data

groupby()	Group and aggregate
isnull()	Check missing values
fillna()	Fill missing values
dropna()	Remove missing values
unique()	Unique values
value_counts()	Frequency count
read_csv()	Load CSV file
to_csv()	Save CSV file

#Most Important 5 to Memorize

df.head()
df.info()
df.describe()
df.sort_values()
df.groupby()

Transformation:

 Example 1: Create a column based on a condition

import pandas as pd

```
df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Charlie"],
    "Marks": [85, 45, 72]
})

df["Result"] = df["Marks"].apply(
    lambda x: "Pass" if x >= 50 else "Fail"
)

print(df)


Output:
text
      Name  Marks Result
0    Alice     85   Pass
1      Bob     45   Fail
2  Charlie     72   Pass


---

 Example 2: Conditional salary increase
```

```
df["Salary"] = df["Salary"].apply(
    lambda x: x * 1.1 if x > 50000 else x
)
```

Meaning:

* Salary > 50000 → increase by 10%
* Otherwise → keep the same

 Example 3: Using `loc` (very common)

```
df.loc[df["Marks"] >= 50, "Result"] = "Pass"
df.loc[df["Marks"] < 50, "Result"] = "Fail"
```

This is often preferred for DataFrames.

 Example 4: Multiple conditions

```
df["Grade"] = df["Marks"].apply(
    lambda x: "A" if x >= 80
    else "B" if x >= 60
    else "C"
)
```

For Marks `[85, 45, 72]`, output:

```
text
Marks  Grade
85     A
45     C
72     B
```

```
df["NewColumn"] = df["ExistingColumn"].apply(
    lambda x: value_if_true if condition else value_if_false
)
```

Example:

```
df["Status"] = df["Age"].apply(
    lambda x: "Adult" if x >= 18 else "Minor"
)
```

This combines transformation (creating/modifying a column) with conditions.